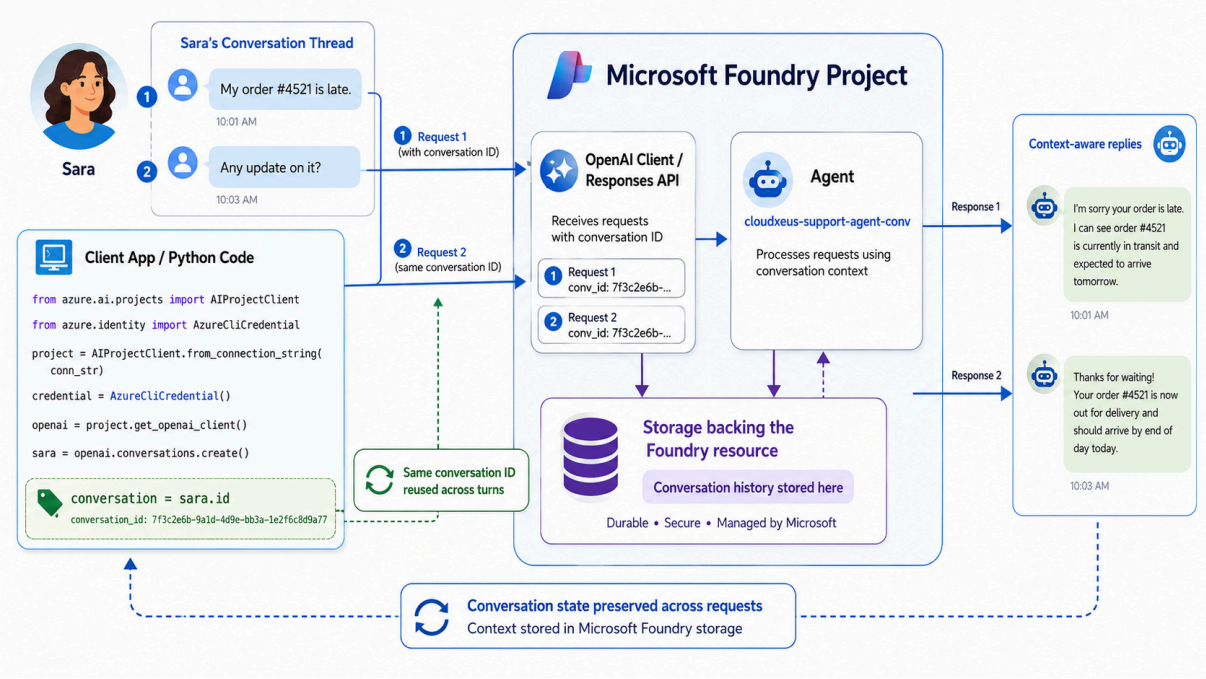
Production-Ready AI Memory, Workflows, Monitoring, and Safety

Review of conversations and memory

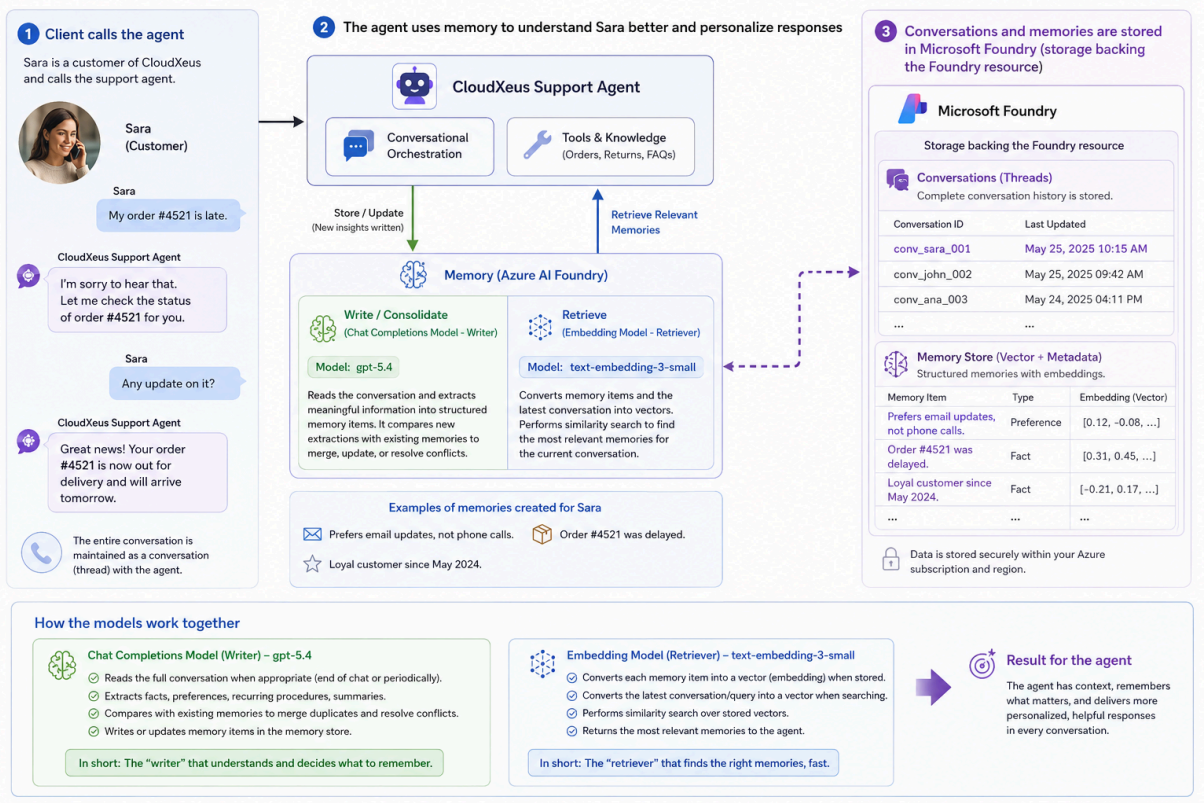


We looked at how conversation state and memory help a Microsoft Foundry agent provide context-aware responses across multiple user messages.

The first part shows a conversation thread with Sara. Sara first says that her order `#4521` is late. A few minutes later, she asks, “Any update on it?” The important point is that the second message does not repeat the order number. The agent can still understand what Sara is referring to because the same conversation ID is reused across turns.

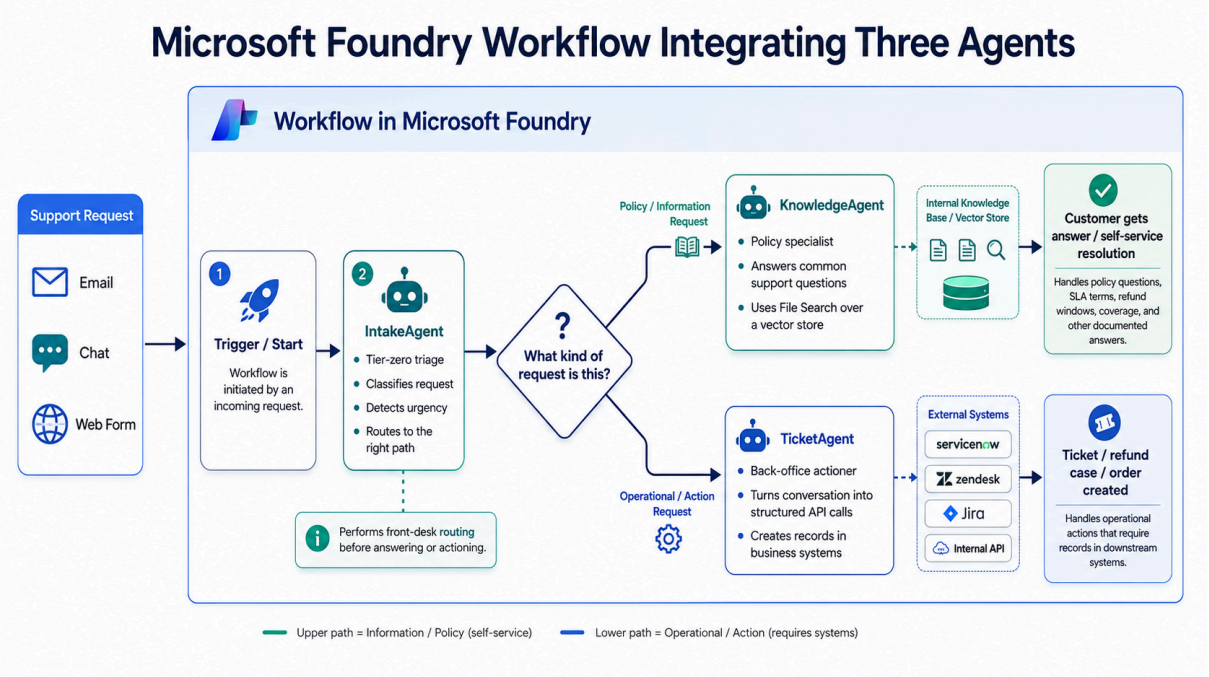
In the code, the application creates a conversation and stores the conversation ID. When the next request is sent, the same conversation ID is passed again. This allows Microsoft Foundry to preserve the conversation thread, so the agent can use earlier messages as context instead of treating each request as a brand-new interaction.

The conversation history is stored in the storage backing the Microsoft Foundry resource. This means the conversation state is durable, secure, and managed by Microsoft. The agent can then provide context-aware replies, such as understanding that “it” refers to order `#4521`.



Conversation history helps within a thread, but memory helps the agent understand the user better over time. For example, the agent may learn that Sara prefers email updates, that order `#4521` was delayed, or that she has been a loyal customer since May 2024.
Memory uses two types of models. A chat completion model, such as GPT 5.4, acts as the writer. It reads the conversation and decides what useful information should be stored as memory. An embedding model, such as `text-embedding-3-small`, acts as the retriever. It converts memory items into vectors so the most relevant memories can be found later.
When Sara contacts the support agent again, the agent can retrieve relevant memories and use them to personalize the response. This allows the agent to respond with more continuity, instead of asking the same questions again or forgetting previous interactions.

Lab - Creating the CloudXeus Agents


Microsoft Foundry Workflow Integrating Three Agents
Workflow in Microsoft Foundry
Support Request
Email
Chat
Web Form
1
Trigger / Start
Workflow is initiated by an incoming request.
2
IntakeAgent
Tier-zero triage
Classifies request
Detects urgency
Routes to the right path
Performs front-desk routing before answering or actioning.
What kind of request is this?
Policy / Information Request
KnowledgeAgent
Policy specialist
Answers common support questions
Uses File Search over a vector store
Internal Knowledge Base / Vector Store
Customer gets answer / self-service resolution
Handles policy questions, SLA terms, refund windows, coverage, and other documented answers.
Operational / Action Request
TicketAgent
Back-office actioner
Turns conversation into structured API calls
Creates records in business systems
External Systems
servicenow
zendesk
Jira
Internal API
Ticket / refund case / order created
Handles operational actions that require records in downstream systems.
Upper path = Information / Policy (self-service)
Lower path = Operational / Action (requires systems)

In the next set of chapters, we are looking at how a Microsoft Foundry workflow can coordinate multiple agents to handle different types of support requests.

The workflow starts when a support request comes in. That request could arrive through email, chat, or a web form. The important point is that the workflow gives us a structured way to handle the request instead of sending every message to the same agent.

The first step is the trigger or start point. This is what begins the workflow when a new conversation or request is received. Once the workflow starts, the request is passed to the IntakeAgent.

The IntakeAgent acts like the front-desk triage agent. Its job is not necessarily to solve the entire request. Instead, it reads the customer's message, classifies the request, checks whether it looks urgent, and decides which path the request should follow. This is similar to how a real support team first determines whether a customer is asking a simple policy question or whether they need an operational action.

The decision point in the middle asks: "What kind of request is this?" This is where the workflow branches into different paths.

The upper path is for policy or information requests. For example, the customer may ask about refund rules, SLA terms, coverage windows, password reset guidance, or other documented information. In this case, the request is routed to the KnowledgeAgent.

The KnowledgeAgent is the policy and knowledge specialist. It can use internal knowledge sources, such as files or a vector store, to answer common support questions. This is useful for self-service scenarios because the customer can get an answer without creating a ticket or involving a back-office team.

The lower path is for operational or action-based requests. For example, the customer may need a refund case created, an order checked, a service request opened, or a record updated in another business system. In this case, the request is routed to the TicketAgent.

The TicketAgent is responsible for turning the conversation into a structured action. It may extract key details from the customer's message, prepare the required information, and interact with external systems such as ServiceNow, Zendesk, Jira, or an internal API. This is

where the workflow moves beyond answering questions and starts taking action.

Agent details

1. wf-IntakeAgent

Instructions

> You classify incoming CloudXeus customer support requests. Decide whether the request is a policy question or a service issue, and whether it relates to a refund. Respond with only the classification.

2. wf-KnowledgeAgent

Instructions

> You answer CloudXeus policy questions using only the company's internal policy documents. If the documents don't contain the answer, say so.

3. wf-TicketAgent

Instructions

> You are the CloudXeus ticketing system. When asked to create a ticket, generate a ticket record with a ticket ID in the format CX-XXXXX, the category, a one-line summary, and a severity from 1 to 4. Respond with only the ticket record.

Lab - Capturing IntakeAgent's Output as Structured Data

JSON Schema for IntakeAgent

```
{
  "name": "triage_result",
  "strict": true,
  "schema": {
    "type": "object",
    "properties": {
      "category": {
        "type": "string",
```

```
        "enum": ["policy_question", "service_issue"],
        "description": "The type of customer support reques
t."
      },
      "refund_related": {
        "type": "boolean",
        "description": "True if the request involves a refu
nd, billing dispute, cancellation refund, or repayment conc
ern."
      }
    },
    "required": ["category", "refund_related"],
    "additionalProperties": false
  }
}
```

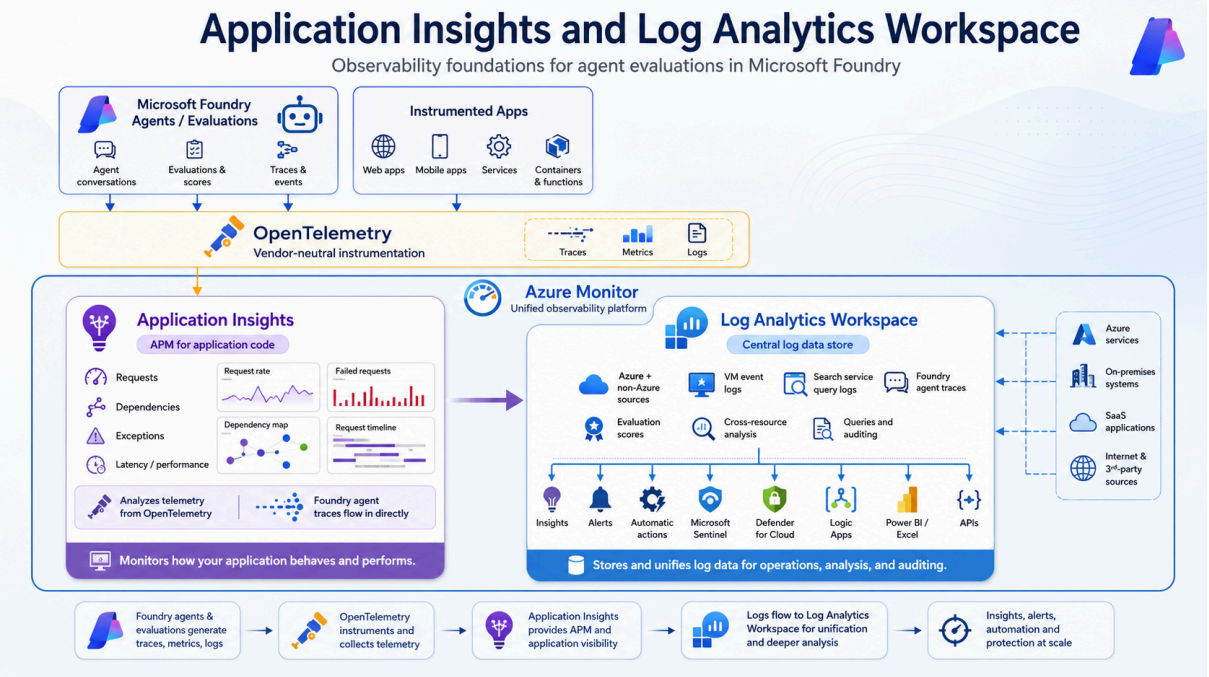
Lab - Adding conditions to the workflow

YAML code for the workflow

```yaml
kind: workflow
trigger:
  kind: OnConversationStart
  id: trigger_wf
  actions:
    - kind: InvokeAzureAgent
      id: node-Intake
      agent:
        name: wf-IntakeAgent
      conversationId: =System.ConversationId
      input:
        messages: =System.LastMessage.Text
      output:
        autoSend: false
        responseObject: Local.Triage
    - kind: ConditionGroup
      id: node-RouteByTriage
```

```
      displayName: Route based on intake classification
      conditions:
        - condition: =Local.Triage.category = "policy_quest
ion"
          id: route_policy_question
          actions:
            - kind: InvokeAzureAgent
              id: node-Knowledge
              agent:
                name: wf-KnowledgeAgent
              conversationId: =System.ConversationId
              input:
                messages: =System.LastMessage.Text
              output:
                autoSend: true
      elseActions:
        - kind: InvokeAzureAgent
          id: node-Ticket
          agent:
            name: wf-TicketAgent
          conversationId: =System.ConversationId
          input:
            messages: =System.LastMessage.Text
          output:
            autoSend: true
    - kind: EndConversation
      id: node-EndConversation
id: ""
name: wf-Triage
description: ""
```

Lab - Enabling Continuous Evaluation on an Agent - Adding Application Insights


Application Insights and Log Analytics Workspace
Observability foundations for agent evaluations in Microsoft Foundry
Microsoft Foundry Agents / Evaluations
Agent conversations
Evaluations & scores
Traces & events
Instrumented Apps
Web apps
Mobile apps
Services
Containers & functions
OpenTelemetry
Vendor-neutral instrumentation
Traces
Metrics
Logs
Azure Monitor
Unified observability platform
Application Insights
APM for application code
Requests
Dependencies
Exceptions
Latency / performance
Request rate
Failed requests
Dependency map
Request timeline
Analyzes telemetry from OpenTelemetry
Foundry agent traces flow in directly
Monitors how your application behaves and performs.
Log Analytics Workspace
Central log data store
Azure + non-Azure sources
VM event logs
Search service query logs
Foundry agent traces
Evaluation scores
Cross-resource analysis
Queries and auditing
Insights
Alerts
Automatic actions
Microsoft Sentinel
Defender for Cloud
Logic Apps
Power BI / Excel
APIs
Stores and unifies log data for operations, analysis, and auditing.
Azure services
On-premises systems
SaaS applications
Internet & 3rd-party sources
Foundry agents & evaluations generate traces, metrics, logs
OpenTelemetry instruments and collects telemetry
Application Insights provides APM and application visibility
Logs flow to Log Analytics Workspace for unification and deeper analysis
Insights, alerts, automation and protection at scale

When we build AI applications or agents, it is not enough to only check whether the model gives an answer. We also need to understand what happened behind the scenes, how the application performed, whether errors occurred, and how the agent behaved over time.

Microsoft Foundry agents and evaluations can generate useful signals such as agent conversations, evaluation scores, traces, and events. At the same time, our own applications — web apps, mobile apps, backend services, containers, and functions — can also generate telemetry. This telemetry helps us understand the health and behavior of the complete AI solution.

OpenTelemetry sits in the middle as the instrumentation layer. It is a vendor-neutral way of collecting telemetry such as traces, metrics, and logs. In simple terms, OpenTelemetry helps capture what the application is doing and sends that observability data into Azure Monitor services.

Application Insights is the application monitoring side of Azure Monitor. It is used to monitor how an application behaves and performs. For example, it can track requests, dependencies, exceptions, failures, response times, and performance issues. Microsoft describes Application Insights as an application performance monitoring feature of Azure Monitor that supports OpenTelemetry-based observability for applications.

So, in an AI application, Application Insights helps us answer questions such as: Is the app receiving requests? Are any API calls failing? How long are model calls taking? Are there exceptions in the code? Are agent traces being captured correctly? This is especially useful when debugging Foundry agents, model calls, tool calls, and backend services.

Log Analytics Workspace is the central log data store. Microsoft describes it as a data store where you can collect log data from Azure resources, non-Azure resources, applications, and other sources. This means it acts as the place where logs and telemetry can be stored, queried, analyzed, audited, and correlated across the environment.

The important difference is this: Application Insights gives us application-level monitoring and performance visibility, while Log Analytics Workspace gives us a centralized place to store and query

logs from many sources. Application Insights can send or use log data through a workspace, and Log Analytics lets us run queries across that data for deeper investigation.